

Tv = v READING GROUP

# Categorical types and AGI

## 1. Categories and Functors

Akshay Shanker

*"Applied category theory is information plumbing. It's boring... but plumbers save more lives than doctors."*

DisCoPy project, "Why?" · [discopy.org](https://discopy.org).

PART 1

# Introduction

## Objective

These sessions run over six to eight talks. Their objective is enough basic working fluency in **category theory** and **type theory** arguments to assess the usefulness of both languages for three subjects:

- applied high-dimensional dynamic programming and reinforcement learning;
- formal representation of models on the computer; and
- AI systems that read, compare, transform, or *verify* those representations.

**Overarching question.** Can categorical type theory formalize structures that are too **complicated**, or that remain **implicit**, in the notation of fields like analysis, calculus, and optimisation?

## Category theory

- Category theory organizes objects and morphisms with specified domains and codomains.
- Category theory shifts the focus from properties of objects to relations between objects and to transformations of those relations.
- Standard results can be proved by simpler arguments that are general enough to be reused across fields.

## Type theory

- Type theory is a formal system that classifies objects by their types.
- Types classify terms (expressions) and rule out invalid compositions among them.
- A **typing judgment**  $\Gamma \vdash t : A$  asserts that, under the assumptions recorded in the **context**  $\Gamma$  — a finite list of typed variables  $x_1 : A_1, \dots, x_n : A_n$  — the **term**  $t$  has the **type**  $A$ . The context, the term, and the type are three different pieces of syntax; the judgment is the assertion relating them.

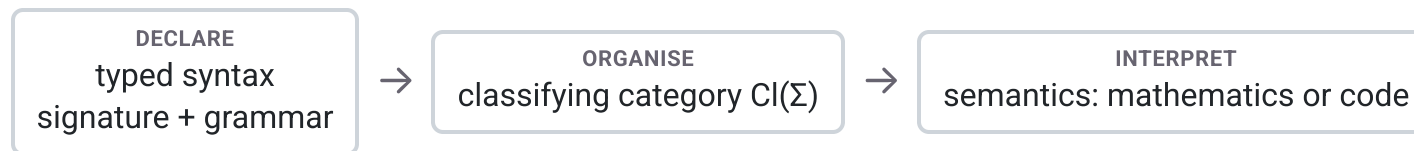
**Categorical type theory.** A type theory generates a classifying category, which records what can be composed. A model is a structure-preserving functor from that category into a **semantic** category.

# Symbolic dynamic programming

The motivating research is a formal symbolic system that represents abstract dynamic programs and recursive decision processes.

- Its syntax is programmable.
- The system represents the abstract model and maps it to computational implementations.
- It does not represent computational procedures.
- Its semantics is denotational (what expressions mean) rather than operational (how a machine executes them).

A closely related project is a functional-programming system for dynamic programming in the style of Backus (1978).



## Contents

1. **Introduction.** Part 1 states the objective, introduces category theory and type theory, and presents the symbolic-dynamic-programming motivation.
2. **Categories and functors.** Part 2 develops Riehl §§1.1–1.3 and §1.6: categories and their examples, isomorphism, commutative diagrams and a diagram chase, duality with Exercise 1.2.vii, monomorphisms and epimorphisms, functors, and the first lemma.
3. **A dynamic-programming application.** Part 3 asks how the Bellman operator is elaborated from first-order typed syntax.



## Notation

- Categories are sans-serif ( $\mathbf{C}, \mathbf{D}$ ); objects are lower case ( $x, y, c$ ); functors are upper case ( $F, G, U$ ); natural transformations are Greek ( $\alpha, \beta, \eta$ ). The collection of morphisms  $x \rightarrow y$  is written  $\mathbf{C}(x, y)$ .
- $\alpha, \beta, \eta$  are reserved for natural transformations; the shock in the application section is written  $\xi'$ , never  $\eta$ .
- $\Gamma$  always denotes a typing context, as in the judgment  $\Gamma \vdash t : A$ ; the feasibility correspondence of the application section is written  $\mathcal{D}$ , with value  $\mathcal{D}(m)$  at the state  $m$ .
- $\mathbf{Cl}(\Sigma)$  denotes the classifying category of a signature  $\Sigma$  (Jacobs 1999). In the application section,  $\mathbb{R}^X$  denotes the bounded measurable real-valued functions on  $X$ , and  $\mathbb{T} : \mathbb{R}^X \rightarrow \mathbb{R}^X$  is the Bellman operator.
- $\llbracket \cdot \rrbracket$  denotes the first-order denotation map of the application section;  $\Upsilon$  denotes the full elaboration, which applies  $\llbracket \cdot \rrbracket$  and then the operator lift; the elaboration of the operator declaration is written  $\Upsilon(\text{op bellman}) = \mathbb{T}$ .
- Scoped reuse, declared once: in the category-theory sections  $f, g, h, k$  are generic morphisms (Riehl's convention); in the application section  $g : X \times A \times Z \rightarrow X$  is the declared state transition.  $P$  denotes a poset  $(P, \leq)$ .

## Preview

The destination of these sessions is the notion of a natural transformation, which needs one prior notion. A **functor**  $F : \mathbf{C} \rightarrow \mathbf{D}$  assigns to each object  $c$  of  $\mathbf{C}$  an object  $Fc$  of  $\mathbf{D}$  and to each morphism  $f : c \rightarrow c'$  a morphism  $Ff : Fc \rightarrow Fc'$ , preserving identities and composition. For parallel functors  $F, G : \mathbf{C} \rightarrow \mathbf{D}$ , a **natural transformation**  $\alpha : F \Rightarrow G$  has a **component**  $\alpha_c : Fc \rightarrow Gc$  for every object  $c$  of  $\mathbf{C}$ , and for every morphism  $f : c \rightarrow c'$  it satisfies

$$\alpha_{c'} \circ Ff = Gf \circ \alpha_c.$$

The later sessions define each part in full.

PART 2

# Categories and functors

## A category

**Definition 1.1.1.** A category consists of a collection of **objects**  $X, Y, Z, \dots$  and a collection of **morphisms**  $f, g, h, \dots$  such that each morphism has a specified domain and codomain ( $f : X \rightarrow Y$ ), each object has an identity  $\text{id}_X : X \rightarrow X$ , and each composable pair has a specified composite –  $f : X \rightarrow Y$  and  $g : Y \rightarrow Z$  yield  $gf : X \rightarrow Z$  – subject to two axioms:

- **unitality:**  $\text{id}_Y f = f = f \text{id}_X$  for every  $f : X \rightarrow Y$ ;
- **associativity:**  $h(gf) = (hg)f$  for every composable triple.

Definition 1.1.1 supplies no elements, no membership relation, and no underlying sets. The objects are recoverable from the identity morphisms (Remark 1.1.2), so of the two collections the **morphisms** take primacy. A category is an algebra of composition.

## Categories versus sets

In each of the examples listed in Example 1.1.3, the collection of objects is not a set (Remark 1.1.5).

**Notation.** For objects  $x, y$ , write  $\mathbf{C}(x, y)$  for the collection of morphisms  $x \rightarrow y$ .

**Definitions 1.1.6–1.1.7.** A category is **small** if it has only a set's worth of arrows in total, and **locally small** if between any pair of objects there is only a set's worth of morphisms — that is, each  $\mathbf{C}(x, y)$  is a set. Small implies locally small, because a subcollection of a set is a set. The converse fails.

A set is known through its elements and the membership relation. A category is known through its morphisms and their composition. Its notion of sameness is isomorphism (Definition 1.1.10), not equality.

## Morphisms need not be functions

### The morphisms are functions

- The objects of  $\mathbf{Set}$  are sets, and its morphisms are functions.
- The objects of  $\mathbf{Vect}_k$  are vector spaces over a field  $k$ , and its morphisms are linear maps.
- The objects of  $\mathbf{Meas}$  are measurable spaces, and its morphisms are measurable functions.
- The objects of  $\mathbf{Poset}$  are partially ordered sets, and its morphisms are order-preserving maps.

### The morphisms are not functions

- The objects of  $\mathbf{Mat}_{\mathbb{R}}$  are positive integers, its morphisms  $n \rightarrow m$  are the  $m \times n$  matrices, and composition is matrix multiplication.
- The category  $\mathbf{BM}$  has a single object. Its morphisms are the elements of a monoid  $M$ , a set carrying an associative multiplication with a unit, and composition is that multiplication.
- A preorder  $(P, \leq)$  is a category with exactly one morphism  $x \rightarrow y$  when  $x \leq y$  and none otherwise. Transitivity supplies the composites, and reflexivity supplies the identities.

## Isomorphism

**Definition 1.1.10.** A morphism  $f : X \rightarrow Y$  is an **isomorphism** when there exists  $g : Y \rightarrow X$  with  $gf = \text{id}_X$  and  $fg = \text{id}_Y$ ; the objects are then isomorphic,  $X \cong Y$ .

### Examples 1.1.11.

- In **Set**, the isomorphisms are the bijections.
- In **Top**, the category whose objects are topological spaces and whose morphisms are continuous maps (Example 1.1.3(ii)), the isomorphisms are the homeomorphisms, a strictly stronger property than being bijective and continuous. The ambient category, not the underlying sets, decides what counts as the same.
- In a poset, the only isomorphisms are the identities, which is the categorical statement of antisymmetry.

*"A category provides a context in which to answer the question  
'When is one thing the same as another thing?'"*

Riehl (2016), *Category Theory in Context*, §1.1, p. 7.



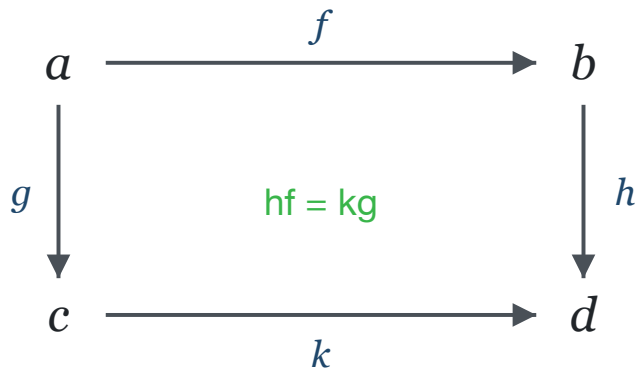
## Commutative diagrams

**Definition 1.6.4.** A **diagram** in a category  $\mathcal{C}$  is a functor  $F : \mathbf{J} \rightarrow \mathcal{C}$ ; the domain  $\mathbf{J}$  is called the indexing category of the diagram.

A **quiver** is a directed graph that may contain loops and parallel arrows. The objects and morphisms of a category form one, and every finite directed path in it has a specified composite, well defined by the associativity axiom of Definition 1.1.1 (pp. 3–4). A diagram is displayed as a quiver of morphisms, and the display **commutes** when the parallel directed paths it presents with common source and target have equal composites. A commutative triangle asserts that the hypotenuse equals the composite of the legs (1.6.1).

## Commutative diagrams

A square indexed by  $2 \times 2$  (Example 1.6.6, Remark 1.6.7) commutes when  $hf = kg$ :



**Lemma 1.6.5.** Functors preserve commutative diagrams.

**Proof.** A diagram in  $\mathbf{C}$  is a functor  $F : \mathbf{J} \rightarrow \mathbf{C}$  (Definition 1.6.4); for any functor  $G : \mathbf{C} \rightarrow \mathbf{D}$ , the composite  $GF : \mathbf{J} \rightarrow \mathbf{D}$  defines the image of the diagram in  $\mathbf{D}$ . ■

## A diagram chase

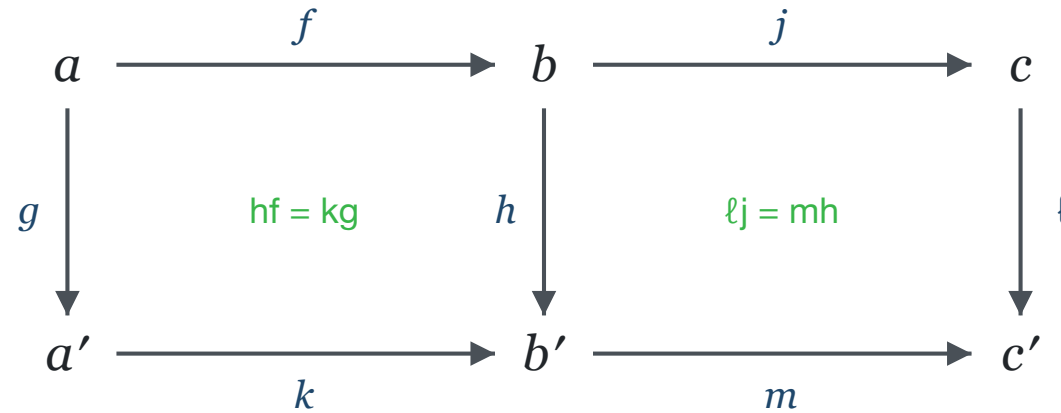
**Lemma 1.6.11.** If, inside a composable path  $f_n, \dots, f_1$ , a segment satisfies  $f_k \cdots f_i = g_m \cdots g_1$ , then  $f_n \cdots f_1 = f_n \cdots f_{k+1} g_m \cdots g_1 f_{i-1} \cdots f_1$ .

**Proof.** Composition is well-defined: if two composites define the same arrow, then pre- and postcomposing each with the same sequences of arrows again gives the same arrow. ■

"This very simple result underlies most proofs by 'diagram chasing'." For a diagram depicted by a **simple acyclic quiver** — at most one edge between any two vertices, and no directed cycles — under the convention that the quiver represents a poset category, commutativity of the entire diagram follows from commutativity of its minimal subdiagrams; for the cube  $2 \times 2 \times 2$ , these are the six faces.

## Pasting squares

**Example (pasting, diagram 1.6.10).** Suppose the two inner squares commute,  $hf = kg$  and  $\ell j = mh$ . Then the outer rectangle commutes,  $\ell(jf) = (mk)g$ :



**Chase.**  $\ell j f = (mh)f$  – substitute  $\ell j = mh$  (Lemma 1.6.11);  $= m(hf)$  – associativity (Definition 1.1.1);  $= m(kg)$  – substitute  $hf = kg$  (Lemma 1.6.11);  $= (mk)g$  – associativity. ■

## Duality through the opposite category

**Definition 1.2.1.** The opposite category  $\mathbf{C}^{\text{op}}$  has the same objects as  $\mathbf{C}$  and a morphism  $f^{\text{op}} : y \rightarrow x$  for each  $f : x \rightarrow y$  of  $\mathbf{C}$ , with composites  $f^{\text{op}} g^{\text{op}} = (gf)^{\text{op}}$ .

**The duality principle.** A theorem of the form "for all categories  $\mathbf{C}$ , a given statement holds" applies in particular to every opposite category  $\mathbf{C}^{\text{op}}$ . Re-expressing that conclusion in terms of the data of  $\mathbf{C}$  reverses the direction of every morphism and the order of every composite. The re-expressed statement is the dual statement, and the re-expressed proof is the dual proof — "a two-for-one deal: any proof in category theory simultaneously proves two theorems" (p. 10).

## Duality through the opposite category

**Lemma 1.2.3.** For  $f : x \rightarrow y$  the following are equivalent: (i)  $f$  is an isomorphism; (ii) postcomposition  $f_* : \mathbf{C}(c, x) \rightarrow \mathbf{C}(c, y), h \mapsto fh$ , is a bijection for every  $c$ ; (iii) precomposition  $f^* : \mathbf{C}(y, c) \rightarrow \mathbf{C}(x, c), k \mapsto kf$ , is a bijection for every  $c$ .

Riehl proves (i)  $\Leftrightarrow$  (ii) directly and obtains (i)  $\Leftrightarrow$  (iii) by re-reading (i)  $\Leftrightarrow$  (ii) in  $\mathbf{C}^{\text{op}}$ . The next two slides work that dualization in full. It is the one worked dualization of these sessions, cited thereafter.

## Applying duality to Lemma 1.2.3

Fix  $f : x \rightarrow y$  in  $\mathbf{C}$ ; by Definition 1.2.1 it corresponds to  $f^{\text{op}} : y \rightarrow x$  in  $\mathbf{C}^{\text{op}}$ . The equivalence (i)  $\Leftrightarrow$  (ii) of Lemma 1.2.3 is proved for all categories, so it holds in  $\mathbf{C}^{\text{op}}$ . Three translations, each an instance of Definition 1.2.1, convert that statement into (i)  $\Leftrightarrow$  (iii) for  $\mathbf{C}$ .

First,  $\mathbf{C}^{\text{op}}(c, y) = \mathbf{C}(y, c)$  for every object  $c$ : a morphism  $c \rightarrow y$  of  $\mathbf{C}^{\text{op}}$  is  $h^{\text{op}}$  for exactly one  $h : y \rightarrow c$  of  $\mathbf{C}$ .

Second, postcomposition by  $f^{\text{op}}$  in  $\mathbf{C}^{\text{op}}$  is precomposition by  $f$  in  $\mathbf{C}$ : for  $h : y \rightarrow c$ , the composition rule of Definition 1.2.1 gives  $f^{\text{op}}h^{\text{op}} = (hf)^{\text{op}} = (f^*h)^{\text{op}}$ , so under the first translation  $(f^{\text{op}})_* : \mathbf{C}^{\text{op}}(c, y) \rightarrow \mathbf{C}^{\text{op}}(c, x)$  is the map  $f^* : \mathbf{C}(y, c) \rightarrow \mathbf{C}(x, c)$ .

## Applying duality to Lemma 1.2.3

Third,  $f^{\text{op}}$  is an isomorphism in  $\mathbf{C}^{\text{op}}$  if and only if  $f$  is an isomorphism in  $\mathbf{C}$ : if  $gf = \text{id}_x$  and  $fg = \text{id}_y$  (Definition 1.1.10), the composition rule gives  $f^{\text{op}}g^{\text{op}} = (gf)^{\text{op}} = \text{id}_x$  and  $g^{\text{op}}f^{\text{op}} = (fg)^{\text{op}} = \text{id}_y$ , so  $g^{\text{op}}$  inverts  $f^{\text{op}}$ ; the converse follows by the same argument in  $\mathbf{C}^{\text{op}}$ , whose opposite is  $\mathbf{C}$ .

Substituting the three translations into "(i)  $\Leftrightarrow$  (ii) for  $f^{\text{op}}$  in  $\mathbf{C}^{\text{op}}$ " yields:  $f$  is an isomorphism in  $\mathbf{C}$  if and only if  $f^* : \mathbf{C}(y, c) \rightarrow \mathbf{C}(x, c)$  is a bijection for every object  $c$ . That is the equivalence (i)  $\Leftrightarrow$  (iii) in  $\mathbf{C}$ .

Exercise 1.2.vii, worked next, shows that the supremum–infimum duality of analysis is exactly categorical duality.



## Exercise 1.2.vii

**Exercise 1.2.vii** (Riehl, verbatim). "Regarding a poset  $(P, \leq)$  as a category, define the supremum of a subcollection of objects  $A \in P$  in such a way that the dual statement defines the infimum. Prove that the supremum of a subset of objects is unique, whenever it exists, in such a way that the dual proof demonstrates the uniqueness of the infimum."

**Notation.**  $(P, \leq)$  is the category with a unique morphism  $x \rightarrow y$  exactly when  $x \leq y$  (Example 1.1.4(iii)), and the subcollection is  $A \subseteq P$ .

## Supremum

**Definition.** An object  $s$  is a **supremum** of  $A$  when:

- (i) for every  $a \in A$  there is a morphism  $a \rightarrow s$  – that is,  $s$  is an upper bound of  $A$ ;
- (ii) for every  $u$  admitting a morphism  $a \rightarrow u$  from every  $a \in A$ , there is a morphism  $s \rightarrow u$  – that is,  $s$  maps into every upper bound.

**Dualization.** By Definition 1.2.1, a morphism  $x \rightarrow y$  in  $P^{\text{op}}$  is a morphism  $y \rightarrow x$  in  $P$ . Since  $P$  has a morphism  $y \rightarrow x$  exactly when  $y \leq x$  (Example 1.1.4(iii)), the order of  $P^{\text{op}}$  is the reversed order:  $x \leq y$  in  $P^{\text{op}}$  means  $y \leq x$  in  $P$ .

## Supremum

Now read the two conditions for an object  $i$  of  $P^{\text{op}}$ . Condition (i) asks for a morphism  $a \rightarrow i$  in  $P^{\text{op}}$  for every  $a \in A$ ; by the translation just stated, this says  $i \leq a$  for every  $a \in A$ , so  $i$  is a lower bound of  $A$  in  $P$ . Condition (ii) asks for a morphism  $i \rightarrow u$  in  $P^{\text{op}}$ , that is,  $u \leq i$  in  $P$ , for every  $u$  that admits a morphism  $a \rightarrow u$  in  $P^{\text{op}}$  from every  $a \in A$ , that is, for every lower bound  $u$  of  $A$  in  $P$ . Together the two conditions say:  $i$  is a lower bound of  $A$ , and every lower bound  $u$  satisfies  $u \leq i$ . By the definition of the infimum as the greatest lower bound,  $i = \inf A$ . The supremum in  $P^{\text{op}}$  is the **infimum** in  $P$ , as the exercise requires.

## Uniqueness of the supremum

**Proposition.** If  $s$  and  $s'$  are both suprema of  $A \subseteq P$ , then  $s = s'$ .

**Proof.** Assume  $s$  and  $s'$  each satisfy conditions (i) and (ii) of the definition of the supremum.

**Step 1.** By condition (i), applied to  $s$  and to  $s'$  in turn, each of  $s$  and  $s'$  is an upper bound of  $A$ .

**Step 2.** Condition (ii) for  $s'$ , applied to the upper bound  $s$  produced by Step 1, yields a morphism  $s' \rightarrow s$ .

**Step 3.** Condition (ii) for  $s$ , applied to the upper bound  $s'$  produced by Step 1, yields a morphism  $s \rightarrow s'$ .

**Step 4.** By Definition 1.1.1 the composites  $s \rightarrow s' \rightarrow s$  and  $s' \rightarrow s \rightarrow s'$  exist. A preorder, and in particular a poset, has at most one morphism between any two objects (Example 1.1.4(iii)), and  $\text{id}_s \in P(s, s)$  by Definition 1.1.1; hence the composite  $s \rightarrow s' \rightarrow s$  equals  $\text{id}_s$ , and likewise  $s' \rightarrow s \rightarrow s'$  equals  $\text{id}_{s'}$ . By Definition 1.1.10 the morphisms of Steps 2 and 3 are therefore mutually inverse isomorphisms, so  $s \cong s'$ .

**Step 5.** In a poset the only isomorphisms are the identities, which is the categorical statement of antisymmetry (Example 1.1.11(v)); the isomorphism of Step 4 is thus an identity, and  $s = s'$ . ■

## Uniqueness by duality

**Duality.** Reading Steps 1–5 in  $P^{\text{op}}$  proves, word for word, that the infimum is unique whenever it exists. No second argument is written.

Steps 1–4 use conditions (i)–(ii), the category axioms, and the fact that a preorder has at most one morphism between any two objects. They do not use antisymmetry. Hence any two suprema of  $A$  in a preorder are isomorphic. In a poset, antisymmetry enters at Step 5 and implies that the two suprema are equal.

## Monomorphisms and epimorphisms

Definitions dualize as theorems do.

**Definition 1.2.7.** A morphism  $f : x \rightarrow y$  in a category  $\mathbf{C}$  is a **monomorphism** if for every object  $w$  and every parallel pair  $h, k : w \rightarrow x$ ,  $fh = fk$  implies  $h = k$ ; and an **epimorphism** if for every object  $z$  and every parallel pair  $h, k : y \rightarrow z$ ,  $hf = kf$  implies  $h = k$ .

**Duality.**  $f : x \rightarrow y$  is a monomorphism in  $\mathbf{C}$  if and only if  $f^{\text{op}} : y \rightarrow x$  is an epimorphism in  $\mathbf{C}^{\text{op}}$ ; and  $f$  is an epimorphism in  $\mathbf{C}$  if and only if  $f^{\text{op}}$  is a monomorphism in  $\mathbf{C}^{\text{op}}$ . To verify the first claim: a parallel pair  $h, k : w \rightarrow x$  in  $\mathbf{C}$  is a parallel pair  $h^{\text{op}}, k^{\text{op}} : x \rightarrow w$  in  $\mathbf{C}^{\text{op}}$ , and the composition rule of Definition 1.2.1 gives  $(fh)^{\text{op}} = h^{\text{op}}f^{\text{op}}$  and  $(fk)^{\text{op}} = k^{\text{op}}f^{\text{op}}$ ; so the left-cancellation property " $fh = fk$  implies  $h = k$ " for  $f$  in  $\mathbf{C}$  is the right-cancellation property " $h^{\text{op}}f^{\text{op}} = k^{\text{op}}f^{\text{op}}$  implies  $h^{\text{op}} = k^{\text{op}}$ " for  $f^{\text{op}}$  in  $\mathbf{C}^{\text{op}}$ , which is the epimorphism condition. The second claim follows by applying the first in  $\mathbf{C}^{\text{op}}$ .

## Monomorphisms and epimorphisms in Set and Ring

**Example 1.2.8.** In **Set**, the monomorphisms are precisely the injections and the epimorphisms are precisely the surjections.

**The axiom of choice.** Let  $f : X \rightarrow Y$  be a surjection. A **section** of  $f$  is a function  $s : Y \rightarrow X$  such that  $f \circ s = \text{id}_Y$  (Example 1.2.9: a right inverse; an epimorphism admitting a section is **split**). The assertion that every surjection in **Set** admits a section is equivalent to the axiom of choice. Because the epimorphisms in **Set** are precisely the surjections (Example 1.2.8), this is Remark 1.2.10's formulation: the axiom of choice asserts that every epimorphism in the category of sets is a split epimorphism.

**Example 1.2.11.** In **Ring**, the inclusion  $\mathbb{Z} \hookrightarrow \mathbb{Q}$  is a monomorphism **and** an epimorphism yet not an isomorphism: there are no ring homomorphisms  $\mathbb{Q} \rightarrow \mathbb{Z}$ . That the inclusion is epic is part of Exercise 1.2.iv; the argument is that a ring homomorphism out of  $\mathbb{Q}$  is determined by its values on  $\mathbb{Z}$ .

Categorical surjectivity is right-cancellability, and right-cancellability can hold without surjectivity. Which morphisms are epimorphisms depends on the ambient category.

## Functors

**Definition 1.3.1.** A functor  $F : \mathbf{C} \rightarrow \mathbf{D}$  assigns an object  $Fc$  to each object  $c$  and a morphism  $Ff : Fc \rightarrow Fc'$  to each  $f : c \rightarrow c'$ , preserving the structure:  $Fg \cdot Ff = F(gf)$  and  $F(\text{id}_c) = \text{id}_{Fc}$ .

### Examples 1.3.2.

- The forgetful functor  $U : \mathbf{Vect}_k \rightarrow \mathbf{Set}$  sends a vector space to its underlying set. The free functor  $F : \mathbf{Set} \rightarrow \mathbf{Group}$  sends a set to the free group on it, whose elements are the reduced words in the set's elements and their formal inverses.
- The fundamental group  $\pi_1 : \mathbf{Top}_* \rightarrow \mathbf{Group}$ , where  $\mathbf{Top}_*$  is the category whose objects are pointed topological spaces — spaces with a chosen basepoint — and whose morphisms are basepoint-preserving continuous maps (Example 1.1.3(iii)). It assigns to a pointed space the group of loops at the basepoint taken up to continuous deformation, and it is the archetype of an invariant.



## Functors

### Examples 1.3.2.

- The chain rule  $D(g \circ f)_a = Dg_{f(a)} \cdot Df_a$  states exactly that  $f \mapsto Df$  is functorial on pointed Euclidean spaces. This is the tenth part of Riehl's Example 1.3.2, where the chain rule is expressed as functoriality of the derivative (pp. 15–16).
- A **contravariant** functor is a functor  $\mathbf{C}^{\text{op}} \rightarrow \mathbf{D}$  (Definition 1.3.5). The closing application slides use one such functor,  $\mathbb{R}^{(-)}$ , which reverses the direction of the state transition.

## Represented functors

*"...every sufficiently good analogy is yearning to become a functor." — John Baez, epigraph to §1.3*

**Definition 1.3.11.** For locally small  $\mathbf{C}$  and  $c \in \mathbf{C}$ : the covariant represented functor  $\mathbf{C}(c, -) : \mathbf{C} \rightarrow \mathbf{Set}$  sends  $x \mapsto \mathbf{C}(c, x)$  and  $f \mapsto f_*$ ; the contravariant  $\mathbf{C}(-, c) : \mathbf{C}^{\text{op}} \rightarrow \mathbf{Set}$  sends  $f \mapsto f^*$ .

Postcomposition is always covariant, and precomposition is always contravariant. The functor  $\mathbb{R}^{(-)}$  of the closing application slides is the measurable-space analogue of the represented functor  $\mathbf{Set}(-, \mathbb{R})$ : it sends a measurable space  $X$  to the bounded measurable functions on  $X$  and a map to precomposition, exactly the pattern of Definition 1.3.11.

Applying "functors preserve isomorphisms" to the represented functors re-proves (i)  $\Rightarrow$  (ii) and (i)  $\Rightarrow$  (iii) of Lemma 1.2.3.

## Additional properties of functors

- **Bifactoriality** (Definitions 1.3.12–1.3.13). The product  $\mathbf{C} \times \mathbf{D}$  is formed componentwise, and the two represented functors combine into one **bifunctor** — a functor of two variables —  $\mathbf{C}(-, -) : \mathbf{C}^{\text{op}} \times \mathbf{C} \rightarrow \mathbf{Set}$ , acting on morphisms by  $g \mapsto hgf$ .
- **Preservation**. Split monomorphisms and epimorphisms — those admitting a one-sided inverse — are preserved, because a one-sided inverse is an equation between composites and functors preserve such equations. General monomorphisms and epimorphisms need not be preserved.
- **Cat**. Small categories and functors form a category  $\mathbf{Cat}$ , which is locally small but not small. The size distinctions of §1.1 return.

## The first lemma of category theory

**Lemma 1.3.8.** *Functors preserve isomorphisms.*

**Proof.** The proof uses Definition 1.1.10 (isomorphism and inverse) and the two functoriality axioms of Definition 1.3.1.

Let  $F : \mathbf{C} \rightarrow \mathbf{D}$  be a functor and  $f : x \rightarrow y$  an isomorphism with inverse  $g : y \rightarrow x$  (Definition 1.1.10). Then

$$Fg \cdot Ff = F(gf) = F(\mathrm{id}_x) = \mathrm{id}_{Fx},$$

the first equality by the composition axiom of Definition 1.3.1, the second because  $gf = \mathrm{id}_x$  (Definition 1.1.10), the third by the identity axiom of Definition 1.3.1. So  $Fg$  is a left inverse of  $Ff$ ; exchanging the roles of  $f$  and  $g$  in the same three equalities gives  $Ff \cdot Fg = \mathrm{id}_{Fy}$ , so the inverse is two-sided and  $Ff$  is an isomorphism (Definition 1.1.10). ■

## Consequences of the first lemma

- If  $\pi_1(X) \not\cong \pi_1(Y)$ , then  $X \not\cong Y$ . Any functorial assignment separates objects that it sends to non-isomorphic values.
- A functor  $\mathbf{B}G \rightarrow \mathbf{C}$ , where  $\mathbf{B}G$  is the one-object category of a group  $G$  (the construction  $\mathbf{B}M$  of the slide "Morphisms need not be functions", applied to a group), is exactly an action of  $G$  (Example 1.3.9). Each  $g \in G$  is an isomorphism in  $\mathbf{B}G$ , so it must act by automorphisms (isomorphisms of an object with itself), with  $(g^{-1})_* = (g_*)^{-1}$ , and no separate proof is required (Corollary 1.3.10).
- The introduction stated that a model of a type theory is a structure-preserving functor from its classifying category. The lemma is the first instance of the pattern these sessions develop, in which a property established for the syntax transfers to every interpretation. The closing application slides work one example in full.

## The Brouwer fixed-point theorem

**Theorem 1.3.3** (Brouwer fixed-point theorem). Any continuous endomorphism of a 2-dimensional disk  $D^2$  has a fixed point.

**The functorial argument.** If a fixed-point-free continuous map  $f : D^2 \rightarrow D^2$  existed, the point where the ray from  $f(x)$  through  $x$  meets the boundary circle would define a retraction  $r : D^2 \rightarrow S^1$  of the boundary inclusion  $i : S^1 \hookrightarrow D^2$ , that is,  $ri = \text{id}_{S^1}$ ; applying  $\pi_1$  (with a basepoint chosen on  $S^1$ ) and the two functoriality axioms of Definition 1.3.1 would give  $\pi_1(r) \pi_1(i) = \pi_1(ri) = \text{id}_{\pi_1(S^1)}$ , making the identity of  $\pi_1(S^1) \cong \mathbb{Z}$  factor through the trivial group  $\pi_1(D^2) \cong 0$ , which is impossible: a homomorphism that factors through the trivial group sends the generator  $1 \in \mathbb{Z}$  to  $0 \neq 1$ .

**Inputs not proved here.** The computations  $\pi_1(S^1) \cong \mathbb{Z}$  and  $\pi_1(D^2) \cong 0$  are inputs to the argument. Riehl obtains them from covering-space theory, which these sessions do not develop.

PART 3

# A dynamic-programming application

## Context-free grammar



Pāṇini, c. 4th century BCE  
the *Aṣṭādhyāyī*



John Backus, 1924–2007  
creator of Fortran; proponent of functional programming

A context-free grammar is a finite set of rewrite rules applied *recursively* to generate every well-formed expression of a language, each rule applying independently of surrounding context. Programming languages are parsed using a grammar (with some context) and then interpreted and compiled.



## First-order syntax for the Bellman operator

$$v(m) = \max_{c \in \mathcal{D}(m)} \left\{ u(c) + \beta \mathbb{E}_{\xi'} v(R(m - c) + \xi') \right\} \quad \text{i.e.} \quad v = \mathbb{T} v$$

- $X$  is the state space, and  $m \in X$  is current resources.  $A$  is the choice space,  $\mathcal{D}(m) \subseteq A$  is the feasible set at  $m$ , and  $c \in \mathcal{D}(m)$  is current consumption.
- $Z$  is the shock space, and  $\xi' \in Z$  is the next-period income shock, drawn from a fixed probability distribution on  $Z$ ;  $\mathbb{E}_{\xi'}$  denotes expectation under that law.
- $R > 0$  is the gross return factor, and  $g : X \times A \times Z \rightarrow X$ , defined by  $g(m, c, \xi') = R(m - c) + \xi'$ , is the transition.
- $u : A \rightarrow \mathbb{R}$  is one-period utility,  $\beta \in (0, 1)$  is the discount factor, and  $v$  is a candidate continuation-value function in  $\mathbb{R}^X$ , the bounded measurable functions on  $X$ .  $\mathbb{T}$  is the Bellman operator, which sends a candidate function  $v$  to a new function  $\mathbb{T}v$  in the same space  $\mathbb{R}^X$ .

## First-order syntax for the Bellman operator

$$v(m) = \max_{c \in \mathcal{D}(m)} \left\{ u(c) + \beta \mathbb{E}_{\xi'} v(R(m - c) + \xi') \right\} \quad \text{i.e.} \quad v = \mathbb{T} v$$

Three assumptions support the display. For each  $m \in X$  the feasible set  $\mathcal{D}(m)$  is nonempty and compact and the objective  $c \mapsto u(c) + \beta \mathbb{E}_{\xi'} v(R(m - c) + \xi')$  is continuous, so the maximum is attained. Because  $\xi'$  has a probability distribution on  $Z$  and  $v$  is bounded and measurable, the expectation is finite. Because  $R(m - c) + \xi' \in X$  for every  $m \in X$ , every  $c \in \mathcal{D}(m)$ , and every realization of  $\xi'$ , the transition returns to the state space.

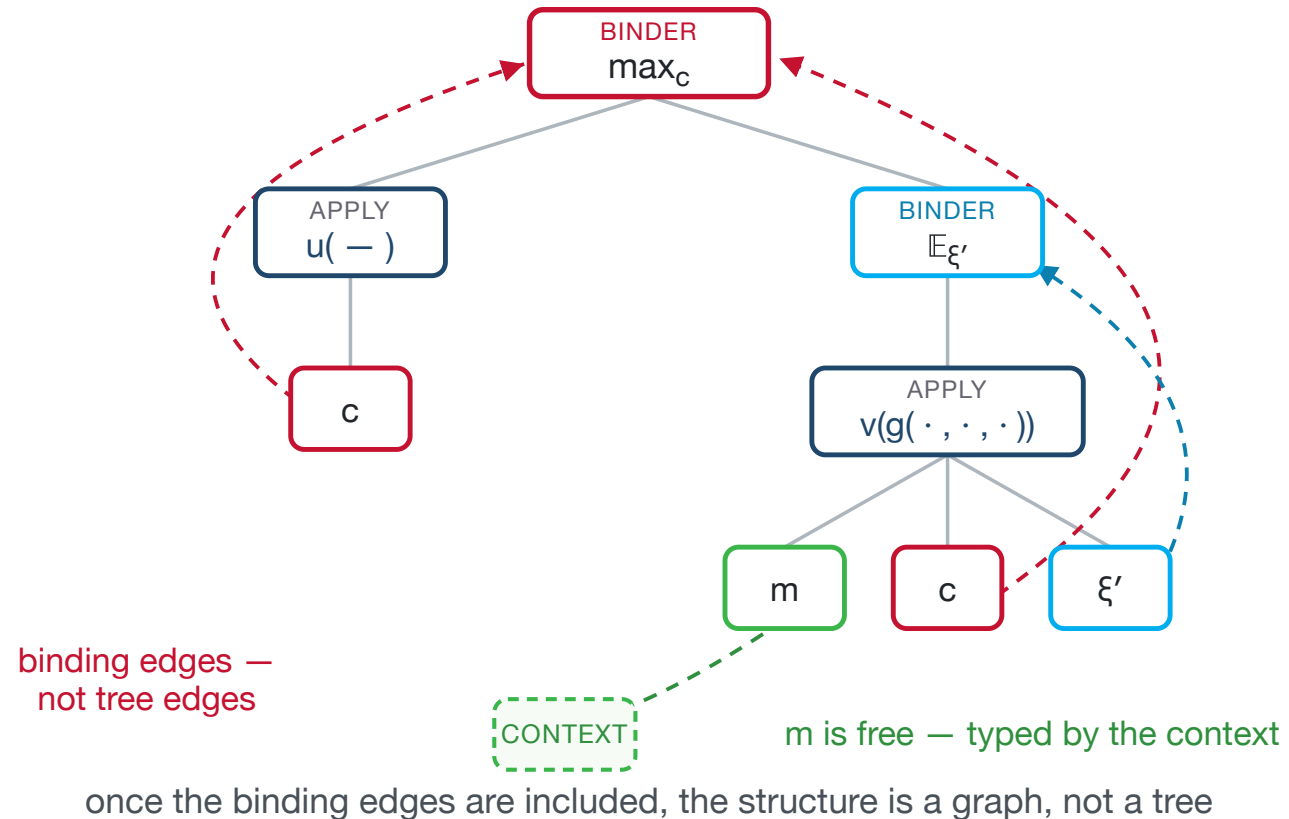
The display quantifies over a function, since  $v$  ranges over  $\mathbb{R}^X$  and  $\mathbb{T}$  maps functions to functions. Call a map **higher-order** when it takes a function as an input or returns one as an output, and call syntax **first-order** when functions are not themselves inputs or outputs of the parsed expressions.

## First-order syntax for the Bellman operator

A parser produces an **abstract syntax tree** (AST): one syntax tree built from the grammar's constructors. The collection of such trees, with those constructor operations, forms the **term algebra** of the grammar's signature (Goguen–Thatcher–Wagner–Wright 1977). **Elaboration** is the construction of the typed semantic object denoted by parsed syntax.

How can first-order typed syntax elaborate the transition and payoff expressions into the higher-order map  $\mathbb{T} : v \mapsto \mathbb{T}v$ ?

# An AST cannot represent bound variables



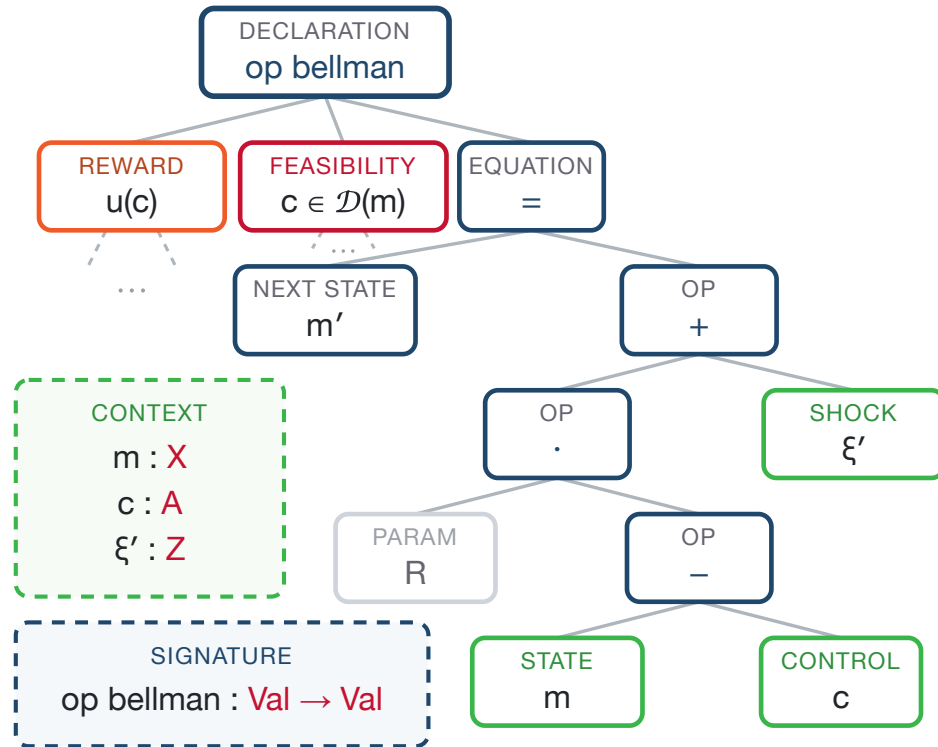
## An AST cannot represent bound variables

Bound occurrences are coloured with their binders ( $c$  with  $\max_c$ ,  $\xi'$  with  $\mathbb{E}_{\xi'}$ ). The variable  $m$  is free and is typed by the context, which the next slide draws in full.

The tree fails in two ways. First, the relation between a binder and its occurrences is not an edge of the tree. The association runs only through the repeated letter  $c$ , and once the relation is drawn as edges the structure is a graph. Second, the distinct trees  $\max_c u(c)$  and  $\max_d u(d)$  denote the same function, so equality of trees is strictly finer than equality of meaning. Higher-order abstract syntax is one representation that repairs both failures.

The abstract syntax tree and the typing context remain necessary. Concrete context-free syntax fixes only which strings parse, so parenthesisation and precedence are artifacts and one meaning has many strings. The tree removes those artifacts. The typing context supplies what no context-free rule expresses. Binding, as the figure shows, needs more still.

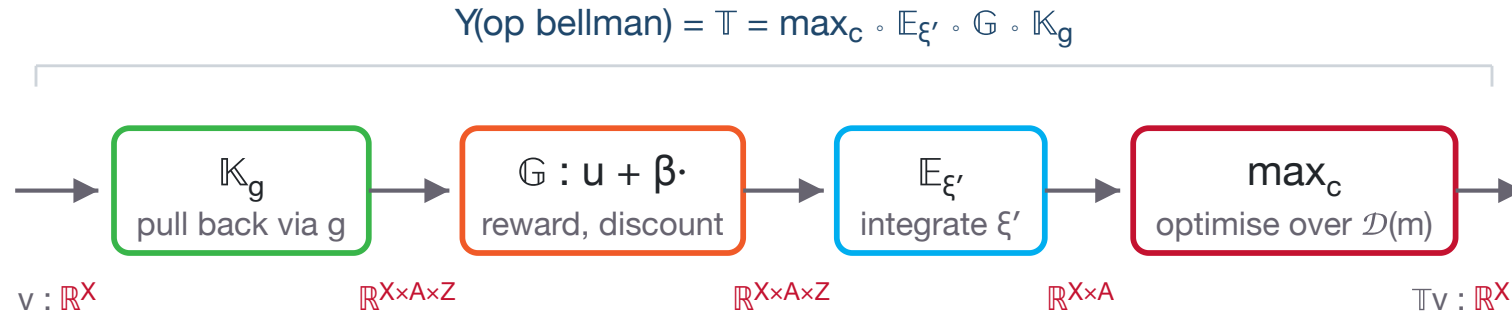
# Elaboration of the Bellman operator



the nodes record bare syntax;  
typing is a judgment relative to the context  
 $(m : X, c : A, \xi' : Z) \vdash R(m - c) + \xi' : X$

① The parser produces the `op bellman` node, which lists first-order equations. The context drawn beside the tree types the leaves, and the signature gives the operator symbol the type  $\text{op bellman} : \text{Val} \rightarrow \text{Val}$ , where  $\text{Val}$  is the signature's sort for value functions. ② The denotation map  $\llbracket \cdot \rrbracket$  sends each equation to its first-order denotation: the arrow  $g$ , the reward  $u$ , and the feasibility correspondence  $m \mapsto \mathcal{D}(m)$ . The syntax trees together form the term algebra of the first-order signature, and  $\llbracket \cdot \rrbracket$  is the unique homomorphism from that term algebra into the first-order semantic algebra, whose carriers are the declared spaces  $(X, A, Z, \mathbb{R}$ , their products, and the subsets of  $A$ , in which the feasibility correspondence takes its values) and whose operations are the declared maps (Goguen–Thatcher–Wagner–Wright 1977).

# Elaboration of the Bellman operator



the boxes are operators; the wire labels are the types of their inputs and outputs

$\llbracket = \rrbracket = g : X \times A \times Z \rightarrow X$  — lifted, reversed, to  $\mathbb{K}_g$   
likewise  $u \rightarrow \mathbb{G}$  and  $\mathcal{D}(\cdot) \rightarrow \max_c$

③ The functor  $\mathbb{R}^{(-)}$  sends each first-order denotation to its operator box, reversing the direction of  $g$ . The reversal is a categorical reading of backward induction. ④ Composition yields  $\Upsilon(\text{op bellman}) = \mathbb{T} : \mathbb{R}^X \rightarrow \mathbb{R}^X$ , realizing the declared signature, with the sort **Val** interpreted as  $\mathbb{R}^X$ . The result is the higher-order object that was never parsed. The boxes apply the reward before the expectation; this order is valid because the aggregator  $w \mapsto u(c) + \beta w$  is affine in  $w$  and the shock distribution has total mass one, so the constant  $u(c)$  passes through  $\mathbb{E}_{\xi'}$ .

# Topics

1. Categories and functors
2. Natural transformations
3. Universal properties and representable functors
4. The Yoneda lemma
5. Limits and colimits
6. Type theory: terms, types, judgments, and contexts
7. Categorical semantics: from typed syntax to models
8. Applications
  - dynamic programming, MDPs, and reinforcement learning
  - precise model descriptions for AI



## Further examples

- The [Category Theory for AGI, Spring 2026 course page](#) is Sridhar Mahadevan's seminar at the University of Massachusetts Amherst.
- [Lecture 2: Functors](#) contains the example in which the objects are Markov decision processes, the arrows are MDP homomorphisms, and reinforcement-learning algorithms are presented as functors to value functions.
- [Categories for AGI](#) is a longer treatment of the MDP and reinforcement-learning example.
- [Universal Decisions with Kan Extensions](#) is a more advanced decision-theory example.

## References

- Backus, John (1978). "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs." ACM Turing Award lecture. *Communications of the ACM* 21(8), 613–641.
- Baez, John (2006). "Quantum Quandaries: A Category-Theoretic Perspective." Cited as the source of the epigraph to Riehl §1.3.
- DisCoPy project. "Why?" [discopy.org](https://discopy.org), accessed 7 August 2026.
- Eilenberg, Samuel, and Saunders Mac Lane (1942). "Natural isomorphisms in group theory." *Proceedings of the National Academy of Sciences USA* 28, 537–543.
- Eilenberg, Samuel, and Norman Steenrod (1952). *Foundations of Algebraic Topology*. Cited as the source of the epigraph to Riehl §1.6.

## References

- Fiore, Marcelo P., Gordon D. Plotkin, and Daniele Turi (1999). "Abstract Syntax and Variable Binding." *Proceedings of the Fourteenth Annual IEEE Symposium on Logic in Computer Science*, 193–202. DOI 10.1109/LICS.1999.782615.
- Goguen, J. A., J. W. Thatcher, E. G. Wagner, and J. B. Wright (1977). "Initial Algebra Semantics and Continuous Algebras." *Journal of the ACM* 24(1), 68–95.
- Ingerman, Peter Zilahy (1967). "'Pāṇini-Backus Form' Suggested." *Communications of the ACM* 10(3), 137. DOI 10.1145/363162.363165.
- Jacobs, Bart (1999). *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141. Amsterdam: Elsevier (North-Holland). ISBN 0-444-50170-3.
- Lamiaux, Thomas, and Benedikt Ahrens (2024). "An Introduction to Different Approaches to Initial Semantics." arXiv:2401.09366 [cs.LO].

## References

- Mahadevan, Sridhar. *Category Theory for AGI*, Spring 2026 course page, University of Massachusetts Amherst. [people.cs.umass.edu/~mahadeva/categorical-agi.html](https://people.cs.umass.edu/~mahadeva/categorical-agi.html), accessed 7 August 2026.
- Mahadevan, Sridhar. "Lecture 2: Functors." [people.cs.umass.edu/~mahadeva/papers/lecture2.pdf](https://people.cs.umass.edu/~mahadeva/papers/lecture2.pdf), accessed 7 August 2026.
- Mahadevan, Sridhar. *Categories for AGI*. [people.cs.umass.edu/~mahadeva/papers/catagi.pdf](https://people.cs.umass.edu/~mahadeva/papers/catagi.pdf), accessed 7 August 2026.
- Mahadevan, Sridhar. *Universal Decisions with Kan Extensions*. [people.cs.umass.edu/~mahadeva/papers/udm.pdf](https://people.cs.umass.edu/~mahadeva/papers/udm.pdf), accessed 7 August 2026.
- Oliveira, Bruno C. d. S., and Andres Löb (2013). "Abstract Syntax Graphs for Domain Specific Languages." *Proceedings of PEPM '13*, 87–96. DOI 10.1145/2426890.2426909.

## References

- Penn, Gerald, and Paul Kiparsky (2012). "On Pāṇini and the Generative Capacity of Contextualized Replacement Systems." *Proceedings of COLING 2012: Posters*, 943–950, Mumbai. [aclanthology.org/C12-2092](https://aclanthology.org/C12-2092).
- Pfenning, Frank, and Conal Elliott (1988). "Higher-Order Abstract Syntax." *Proceedings of PLDI '88*, 199–208. DOI 10.1145/53990.54010.
- Riehl, Emily (2016). *Category Theory in Context*. Dover Publications. Author's free PDF: [emilyriehl.github.io/files/context.pdf](https://emilyriehl.github.io/files/context.pdf).
- Stan Development Team (2025). *Stan Reference Manual*, version 2.39.